

T. VIKAS

2303A52281

Binary Search Tree - 4 Important Questions

class Node:

```
def __init__(self, data):
```

```
    self.data = data
```

```
    self.left = None
```

```
    self.right = None
```

1. Delete a Node from BST

```
def deleteNode(root, key):
```

```
if root is None:
```

```
    return None
```

```
# Search left
```

```
if key < root.data:
```

```
    root.left = deleteNode(root.left, key)
```

```
elif key > root.data:
```

```
    root.right = deleteNode(root.right, key)
```

```
else:
```

```
    if root.left is None:
```

```
        return root.right
```

```
    if root.right is None:
```

```
        return root.left
```

```
    temp = root.right
```

```
    while temp.left:
```

```
        temp = temp.left
```

```
    root.data = temp.data
```

```
    root.right = deleteNode(root.right, temp.data)
```

```
    return root
```

Inorder Traversal

```
def inorder(root):
```

```
    if root:
```

```
        inorder(root.left)
```

```
        print(root.data, end=" ")
```

```
        inorder(root.right)
```

2. Check if Binary Tree is BST

```
def isBST(root):
```

```
    arr = []
```

```
    def inorder_list(node):
```

```
        if node is None:
```

```
            return
```

```
            inorder_list(node.left)
```

```
            arr.append(node.data)
```

```
            inorder_list(node.right)
```

```
    inorder_list(root)
```

```
    for i in range(1, len(arr)):
```

```
        if arr[i] <= arr[i - 1]:
```

```
            return False
```

```
    return True
```

3. Kth Smallest Element

```
def kthSmallest(root, k):
```

```
    arr = []
```

```
    def inorder_list(node):
```

```
        if node is None:
```

```
            return
```

```
            inorder_list(node.left)
```

```
            arr.append(node.data)
```

```
            inorder_list(node.right)
```

```

inorder_list(root)

if k <= 0 or k > len(arr):
    return None

return arr[k - 1]

def kthLargest(root, k):
    arr = []

    def reverse_inorder(node):
        if node is None:
            return

        reverse_inorder(node.right)

        arr.append(node.data)

        reverse_inorder(node.left)

```

```

reverse_inorder(root)

```

```

if k <= 0 or k > len(arr):
    return None

```

```

return arr[k - 1]

```

4. Check if BST is Balanced

```

def isBalanced(root):
    def height(node):
        if node is None:
            return 0

        left_height = height(node.left)
        right_height = height(node.right)

        if abs(left_height - right_height) > 1:
            return -1

    if left_height == -1 or right_height == -1:
        return -1

```

```
        return 1 + max(left_height, right_height)

    return height(root) != -1

if __name__ == "__main__":
    root = Node(50)
```

```
    root.left = Node(30)
```

```
    root.right = Node(70)
```

```
    root.left.left = Node(20)
```

```
    root.left.right = Node(40)
```

```
    root.right.left = Node(60)
```

```
    root.right.right = Node(80)
```

1. Delete Node

```
    print("1. Before Deletion:")
```

```
    inorder(root)
```

```
    print()
```

```
    root = deleteNode(root, 30)
```

```
    print("After Deleting 30:")
```

```
    inorder(root)
```

```
    print()
```

2. Check BST

```
    print("\n2. Is Binary Tree a BST?")
```

```
    if isBST(root):
```

```
        print("Yes")
```

```
    else:
```

```
        print("No")
```

3. Kth Smallest

k = 2

print("\n3. Kth Smallest:")

result = kthSmallest(root, k)

if result is not None:

 print(result)

else:

 print("Invalid k")

3. Kth Largest

print("\nKth Largest:")

result = kthLargest(root, k)

if result is not None:

 print(result)

else:

 print("Invalid k")

4. Check Balanced BST

print("\n4. Is BST Balanced?")

if isBalanced(root):

 print("Yes")

else:

 print("No")